



EAST WEST UNIVERSITY

Mini Project Report

Dynamic Spatial–Temporal Graph Transformer for EEG-Based Valence–Arousal Emotion Recognition on the DEAP Dataset

Course: ICE478

Course Title: Machine Learning

Section: 06

Group: 09

Course Instructor: Dr. Raihan Ul Islam

Team Members:

Name	ID
Tahsina Tabassum	2023-1-50-002
Saima Akter Tisha	2022-3-50-018
Fariya Ahmed Bhuiyan	2022-3-50-009
Iftekhar Alam	2019-2-57-039

Baseline Model Report

Graph Neural Network for EEG Valence Classification (DEAP Dataset)

1. Purpose of This Notebook

This notebook takes the pipeline built during the EDA stage — band-power features extracted from EEG, subject-grouped train/test splits, and a k-nearest-neighbor electrode graph — and trains a first graph neural network on it. The goal here isn't a polished model. It's a baseline: something simple enough to trust, run end to end, and use as a floor that later architectures need to beat. The target is valence, binarized at the midpoint of the 1–9 self-report scale.

2. Data and Graph Recap

The pipeline is unchanged from the EDA notebook, so this section is a short recap rather than a full walkthrough. EEG from 19 of the 32 DEAP subjects was split into 4-second windows, and each window was reduced to a 32×5 feature matrix — one row per electrode, one column per frequency band (theta, alpha, low beta, high beta, gamma), all log-transformed. That gives 11,400 windows total. Subject-grouped splitting sent 15 subjects (9,000 windows) to training and the remaining 4 subjects (2,400 windows) to testing, so no subject's data appears in both sets.

The electrode graph also carries over as-is: 32 nodes placed at approximate 10-20 scalp coordinates, connected to their 5 nearest neighbors by a Gaussian-weighted distance kernel, giving 160 directed edges. This same graph is attached to every window regardless of subject or trial.

Item	Value
Feature shape per window	32 nodes × 5 band-power features
Train / test windows	9,000 / 2,400
Train / test subjects	15 / 4 (disjoint)
Valence balance (train+test)	8,850 low / 2,550 high (≈22% high)
Graph structure	32 nodes, 160 edges (k = 5)

3. Model Architecture

The model is a small custom graph network, not one of PyTorch Geometric's built-in layers. Each GNNLayer is implemented on top of MessagePassing with mean aggregation, and it keeps two separate linear transforms: one applied to a node's own features (`lin_self`) and one applied to the features it receives from its neighbors before aggregation (`lin_neigh`), with each incoming message scaled by the edge weight from the k-NN graph. The two paths are summed, so every layer mixes what a channel already knows with a weighted average of what its five nearest electrodes know.

The full network stacks two of these layers ($5 \rightarrow 32 \rightarrow 32$ hidden units) with ReLU activations and dropout ($p = 0.3$) in between, then pools all 32 node embeddings into a single graph-level vector with global mean pooling, and passes that through a linear head to produce two class logits (low / high valence).

Component	Setting
Input dimension	5 (band-power features per node)
Hidden dimension	32
GNN layers	2 (custom mean-aggregation message passing)
Dropout	0.3
Pooling	Global mean pool over 32 nodes
Output classes	2 (low / high valence)

4. Training Setup

Training used Adam with a learning rate of $1e-3$ and weight decay of $5e-4$, standard cross-entropy loss, and ran for 40 epochs. Training accuracy and loss were logged every epoch; test metrics were checked every 5 epochs. Batch size was 32 for training and 64 for evaluation, both handled through PyTorch Geometric's DataLoader, which batches multiple graphs (windows) together for each forward pass.

Epoch	Train loss	Train acc	Test loss	Test acc
5	0.522	0.773	0.497	0.787
10	0.517	0.773	0.490	0.787
15	0.516	0.773	0.492	0.787
20	0.513	0.774	0.497	0.785
25	0.512	0.774	0.497	0.787
30	0.510	0.774	0.494	0.787
35	0.508	0.773	0.490	0.787
40	0.507	0.773	0.491	0.788

The numbers barely move after epoch 5. Loss creeps down by a few thousandths per five-epoch block and accuracy sits at 77–79% the whole way through. That flatness is the first sign something is off — a model that's actually learning to separate two classes usually shows more movement than this over 40 epochs.

5. Evaluation Results

On the held-out test set (2,400 windows from 4 subjects), the model reached 78.8% accuracy. On its own, that sounds like a reasonable baseline. The macro-F1 score tells a different story: 0.443.

Class	Precision	Recall	F1-score	Support
Low	0.79	1.00	0.88	1,890
High	1.00	0.00	0.00	510

The confusion matrix makes the problem concrete: of 1,890 low-valence windows, all 1,890 were predicted correctly. Of 510 high-valence windows, 509 were predicted as low and exactly 1 was predicted correctly.

Confusion matrix: `[[1890, 0], [509, 1]]`

This is the low-valence class share of the test set almost exactly ($1,890 / 2,400 = 78.75\%$), which is what gives away what happened: the model didn't learn to distinguish valence from EEG features. It learned that predicting "low" every time is the cheapest way to minimize cross-entropy loss on an imbalanced dataset, and it settled into that shortcut almost immediately, which is also why the training curve in Section 4 barely moves after the first few epochs.

6. Why This Happened

The imbalance flagged during EDA — roughly 22% high-valence windows against 78% low-valence — carries directly into training. With plain cross-entropy and no reweighting, always predicting the majority class is a strong local minimum: it costs the model almost nothing in loss and needs no real signal from the graph or the features. Two class labels used here also come from binarizing a continuous 1–9 rating at the midpoint, which is a coarse cut that can put genuinely ambiguous, near-threshold ratings on either side, adding label noise on top of the imbalance.

It's also worth noting the model architecture is not the likely culprit. A two-layer message-passing network with mean pooling is simple, but simple models can still separate imbalanced classes when the loss function is set up to make that worthwhile. Here, nothing in the training setup gives the model a reason to bother with the high-valence windows.

7. Limitations of This Baseline

Accuracy alone hid the collapse for most of this run, which is the main lesson to carry forward: any future comparison against this baseline needs to report macro-F1, per-class recall, or a confusion matrix, not accuracy by itself. A few other limitations are worth flagging before this baseline is used as a comparison point.

- Class imbalance is unaddressed. Weighted cross-entropy, focal loss, or oversampling the minority class would all be reasonable next steps.
- Only 19 of 32 subjects were used, carried over from the earlier feature-extraction cache; the full cohort would give more training signal and a less brittle test split.
- The electrode graph is fixed and identical for every window. It was never learned from the data or validated against known EEG connectivity patterns, so it may not reflect the relationships that actually matter for valence.
- Only valence was modeled here. Arousal, which showed a similar imbalance in the EDA stage, would need the same treatment.
- No hyperparameter search was run — learning rate, hidden size, dropout, and number of layers were all single, untuned choices.

8. Summary

This baseline confirms the data pipeline and graph construction work end to end with PyTorch Geometric's Dataset and DataLoader interfaces, and that the custom GNN layer trains without errors. What it does not yet show is a model that has learned anything about valence from EEG: it has found the shortcut of

predicting the majority class and stopped there. The 78.8% accuracy number should not be reported or compared against without the macro-F1 and per-class recall alongside it. Fixing the class imbalance is the single most useful next step before any architectural changes are worth trying.